# AWS [stephane maarek]

Sign in → IAM user → Account activation → payment method → phone verification → Free plan

Console [Region selection]
↓ which is geographically close to you

- IAM → Identity & Access Management, Global service
  ↓
  Users & Groups

→ Root account created by default, shouldn't be used or shared
→ Users are people within your organization, and can be grouped.
→ Groups only contain users, not other groups.
→ Users don't have to belong to a group, and user can belong to multiple groups.

WHY WE CREATE USERS & GROUPS:
→ users or groups can be assigned JSON documents called policies.
→ These policies define the permissions of the users.

- In AWS we use least priviledge principle.

IAM Policy structure:
→ Consists of version: policy language version, always include "2012-10-17"
→ Id: an identifier for the policy (optional)
→ Statement one or more individual statements (required)
→ Statements consists of
'Sid': an identifier for the statement (optional)
'Effect': whether the statement allows or denies access (Allow, Deny)
→ principal: account/user/role to which this policy applied to

- Action: list of actions this policy allows or denies.
- Resource: list of resources to which the actions applied to
- Condition: conditions for when this policy is in effect (optional)

IAM - Password Policy

- Strong passwords = higher security for your account
- In AWS, can you setup a password policy:
  - set a minimum password length
  - Require specific character types:
    - → including uppercase letters
    - → lowercase letters
    - → numbers
    - → non-alphanumeric characters.
  - Allow all IAM users to change their own passwords
  - Require users to change their password after some time (password expiration)
  - prevent password re-use.

MFA : Multi Factor Authentication -

- Users have access to your account and can possibly change configurations or delete resources in your AWS account
- You want to protect your Root Accounts & IAM users
- MFA = password you know + Security device you own

  password + (MFA) => Successful login

- Main benefit of MFA:

  if a password is stolen or hacked, the account is not compromised.

MFA devices options in AWS

→ Virtual MFA device → [ Google Authenticator
                        Authy (phone only)

support for multiple tokens on a single device

→ Universal 2nd Factor (U2F) security key
Yubikey by Yubico (3rd party)
↳ support for multiple root and IAM users using a single security key

→ Hardware key Fob MFA Device (provided by Gemalto)

→ Hardware key Fob MFA Device for AWS GovCloud (US) (provided by SurePassID (3rd party)

## How Can Users Access AWS?

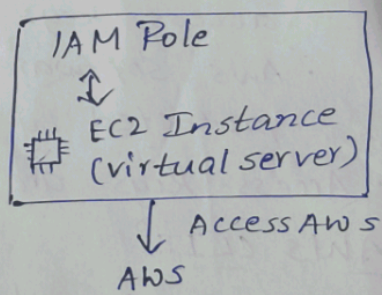
- To access AWS, you have three options:
  - AWS Management Console (protected by password + MFA)
  - AWS Command Line Interface (CLI): protected by access keys.
  - AWS Software Developer kit (SDK) - for code: protected by access keys.
- Access keys are generated through the AWS Consde.

## AWS CLI:

- A tool that enables you to interact with AWS services using commands in your command-line shell
- Direct access to the public APIs of AWS services.
- You can develop scripts to manage your resources.
- It's open-source https://github.com/aws/aws-cli
- Alternative to using AWS Management Console.

AWS SDK:
- AWS software Development Kit (AWS SDK)
- Language-specific APIs (set of libraries)
- Enables you to access and manage AWS services programmatically
- Embedded within your application
- Supports
  - SDK (JavaScript, Python, PHP, .NET, Ruby, Java, GO, Nodejs,...)
  - Mobile SDKs (Android, iOS,...)
  - IOT Device SDKs (Embedded C, Arduino,...)
- Example: AWS CLI is built on AWS SDK for python

IAM Roles for Services:
- Some AWS service will need to perform actions on your behalf
- To do so, we will assign permissions to AWS services with IAM Roles.
- Common roles:
  - $EC_2$ Instance Roles
  - Lambda Function Roles
  - Roles for cloud Formation




IAM Security Tools:
- IAM Credentials Report (account-level)
  → a report that lists all your account's users and the status of their various credentials.
- IAM Access Advisor (user-level)
  → Access advisor shows the service permissions granted to a user and when those services were last accessed.

IAM Guidelines & Best Practices

- Don't use the root account except for AWS account setup

Scalability & High Availability:

- Scalability means that an application/system can handle greater loads by adapting.
- There are two kinds of scalability:
  - Vertical scalability
  - Horizontal scalability (= elasticity)

1. Vertical scalability:
   - → It means increasing the size of the instance for example, your application runs on a t2.micro
   - → Scaling that application vertically means running it on a t2.large.
   - → vertical scalability is very common for non distributed systems, such as a database.
   - → RDS, Elasticache are services that can scale vertically.
   - → There's usally a limit to how much you can vertically scale (hardware limit)

2. Horizontal scalability:
   - Horizontal scalability means increasing the number of instances/systems for your application
   - Horizontal scaling implies distributed systems.
   - This is very common for web applications/ modern applications.
   - It's easy to horizontally scale thanks to the cloud offerings such as Amazon EC2

## High Availability & Scalability for EC2

→ High Availability usually goes hand in hand with horizontal scaling.

→ High availability means running your application/system in at least 2 data centers (== Availability zones)

→ The goal of high availability is to survive a data center loss

→ The high availability can be passive (for RDS Multi AZ for example)

→ The high availability can be active (for horizontal scaling)

→ Vertical Scaling: Increase instance size (= scale up / down)

- From: t2.nano – 0.5 G of RAM, 1 vCPU
- To: u-12tb1.metal – 12.3 TB of RAM, 448 vCPUs

→ Horizontal Scaling: Increase number of instances (= scale out/in)

- Auto Scaling Group
- Load Balancer

→ High Availability: Run instances for the same application across multi AZ

- Auto Scaling Group multi AZ
- Load Balancer multi AZ

## LOAD BALANCING

- Load Balances are servers that forward traffic traffic to multiple servers (e.g., EC2 instances) downstream.

→ Spread load across multiple downstream instances.
→ Expose a single point of access (DNS) to your application.
→ seamlessly handle failures of downstream instances
→ Do regular health checks to your instances
→ provide SSL termination (HTTPS) for your websites.
→ Enforce stickiness with cookies.
→ High availability across zones.
→ separate public traffic from private traffic.

### Elastic Load Balancer:

- An Elastic load Balancer is a managed load Balancer
  - AWS guarantees that it will be working
  - AWS takes care of upgrades, maintenance, high availability
  - AWS provides only a few configuration knobs.

→ It costs less to setup your own load balancer but it will be a lot more effort on your end.
→ It is integrated with many AWS offerings/ services.
  - EC2, EC2 Auto Scaling Groups, Amazon ECS